LithosAnanke Kernel and Capsule System

Volume II of the StarshipOS Technical Reference

Robert A. James

Contents

Chapter 1

LithosAnanke Kernel Architecture

1.1 Design Philosophy

1.2 Boot Sequence

The boot sequence from firmware to FORTH REPL:

1. UEFI Firmware → `uefi_loader.c` (`BOOTX64.EFI`)
2. `ExitBootServices()` — kernel owns hardware; receives `BootInfo{memory map, ACPI, framebuffer}`
3. `kernel_main()` — M1 through M7 milestone initialization:

   M1 Console init (UART 16550 + framebuffer)
   M2 PMM — physical memory manager, bitmap allocator
   M3 VMM — 4-level x86_64 paging
   M4 IDT + APIC interrupts
   M5 TSC + HPET + APIC timer (100 Hz heartbeat)
   M6 `kmalloc` heap (16 MB)
   M7 StarForth VM bootstrap + capsule loading → `ok` REPL

1.3 Milestone Status

Table 1.1: LithosAnanke v1.0.8 milestone status

Milestone	Description	Status
M0–M5	Boot, console, PMM, VMM, IDT/APIC, timer	Complete
M6	`kmalloc` infrastructure	Present (full validation deferred)
M7	StarForth VM bootstrap + capsule execution pipeline	In progress

1.4 Memory Layout

1.5 Source Tree

1.6 Multi-Architecture Support

Table 1.2: Kernel memory layout

Region	Description
Kernel heap	16 MB (`kmalloc`)
Block RAM LBN 0–991	1 MB dedicated RAM blocks
Kernel ramdrive LBN 2048–3071	1 MB for capsule loading
LBN 2048	Entry point for `init.4th`

Listing 1.1: StarKernel build targets

```
make -f Makefile.starkernel              # Build for amd64
make -f Makefile.starkernel ARCH=aarch64
make -f Makefile.starkernel ARCH=riscv64
make -f Makefile.starkernel qemu         # Run in QEMU with OVMF
```

Chapter 2

Capsule System

2.1 Capsule Types

Table 2.1: Capsule types

Type code	Name	Purpose
`(m)`	`MAMA_INIT`	Exactly one Mama VM initialization capsule
`(p)`	`PRODUCTION`	Truth-bearing baby VM initializers
`(e)`	`EXPERIMENT`	DoE workload-only initializers

2.2 Birth Protocol

The capsule birth protocol:

1. Locate capsule by name
2. Validate content hash (XXHash64)
3. Allocate VM ID
4. Execute IDENTITY (capsule code)
5. Execute PERSONALITY (block 1 from ramdrive)
6. Log parity record: VM ID + capsule hash + dict hash

2.3 Content Addressing and Immutability

2.4 Parity Logging

2.5 Capsule Files

2.6 Mama Forth Supervisor

Chapter 3

Word-Level ACL and Security

3.1 ACL System in Kernel Context

3.2 Two-Console Model

3.3 Zuse Superuser

3.4 Future: PKI and Ed25519 (Phase 8)

Chapter 4

Platform Support

4.1 Linux / Hosted

4.2 L4Re / Fiasco.OC

4.3 Bare Metal: amd64

4.4 Bare Metal: aarch64 and riscv64

4.5 Platform Abstraction Layer (HAL Target)

4.6 Raspberry Pi Build

Listing 4.1: Raspberry Pi cross-compilation

```
1 make rpi4-cross     # Cross-compile for Raspberry Pi 4 (ARM64)
```

Chapter 5

L8 Jacquard Mode Selector

5.1 Six Operating Modes

Table 5.1: L8 Jacquard operating modes

Mode	Characteristics
`stable`	—
`volatile`	—
`diverse`	—
`temporal`	—
`transition`	—
`omni`	—

5.2 Attractor Bucket Mechanism

5.3 ssm_jacquard.c Implementation

5.4 Jacquard Capsule Variants

Chapter 6

QEMU Testing and Regression Baseline

6.1 Running the Kernel in QEMU

Listing 6.1: Running LithosAnanke in QEMU

```
make -f Makefile.starkernel qemu
# Requires OVMF UEFI firmware in the OVMF_PATH (see Makefile.
    starkernel)
```

6.2 Regression Baseline

The file `QEMU_BASELINE.log` at the repository root captures the reference QEMU/OVMF output from a known-good LithosAnanke build. Any deviation in a new build's output from this baseline indicates a regression.